FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Amabili Flor

Produto: **BarberCloud SaaS**
https://github.com/glaffamabili/barbercloud-saas

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 10/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 6
3.6 Segurança 7
3.7 Observabilidade, CI/CD e documentação da API 7
3.8 Camadas IaaS, PaaS e SaaS do projeto 8
4 Quadro de entregas 8
5 Atividades realizadas 9
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 9
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 10
5.3 Atividade 9 — Fork e plano de negócio 11
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 13
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 14
6 Considerações finais 16
7 Declaração de autoria 16

1 Introdução

A disciplina de **Introdução à Computação em Nuvem (ISW055)**, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na Fatec Pompeia, tem como propósito formar profissionais capazes de projetar, construir, documentar e operar sistemas distribuídos em nuvem utilizando microsserviços conteinerizados, bancos de dados relacionais e não-relacionais, armazenamento de objetos compatível com S3, controle de acesso baseado em papéis (RBAC), observabilidade e monetização via provedores de pagamento SaaS.

Durante o primeiro bimestre (P1), essa fundação técnica foi construída de forma incremental em torno do **Catálogo de Filmes de Tom Hanks** (https://github.com/glaffamabili/catalogo-tom-hanks). No segundo bimestre (P2), o foco evoluiu da engenharia técnica para a criação de um **produto Software as a Service (SaaS) completo e financeiramente sustentável**: primeiro incorporando cobrança real em modo de teste com o Stripe (Atividade 7) e documentando reversamente toda a arquitetura e camadas de nuvem IaaS/PaaS/SaaS (Atividade 8); em seguida, realizando o espelhamento completo do histórico Git para dar origem ao **BarberCloud SaaS** (https://github.com/glaffamabili/barbercloud-saas) (Atividade 9).

O **BarberCloud SaaS** resolve uma dor real e cotidiana de milhares de barbearias independentes, pequenos salões de beleza e pet shops brasileiros: o caos operacional de gerenciar agendamentos manualmente pelo WhatsApp ou em cadernos de papel, o que gera perda de tempo durante os cortes, conflitos de agenda e alto índice de furos de horário (**no-show**). A plataforma entrega uma vitrine pública de serviços com fotos armazenadas no MinIO S3, motor de agendamento online com isolamento multi-tenant por cliente, três papéis de negócio (`cliente`, `profissional` e `admin`), trilha de eventos de negócio em tempo real no Redis Streams e assinatura recorrente com Stripe (**Plano Free** gratuito até 3 agendamentos ativos e **Plano Pro** por R$ 29,90/mês ilimitado, com Portal do Cliente e painel executivo de MRR).

Este relatório consolidado apresenta na Seção 2 a metodologia de trabalho e aferição de evidências; na Seção 3 a especificação arquitetural completa do **BarberCloud SaaS** em produção; na Seção 4 o quadro síntese de entregas do 2º bimestre; na Seção 5 as fichas técnicas detalhadas das Atividades 7 a 11 com comprovações visuais; e na Seção 6 as reflexões críticas e aprendizados do semestre.

2 Metodologia

As Atividades 7 e 8 foram desenvolvidas e comitadas diretamente no repositório público original do catálogo de filmes (https://github.com/glaffamabili/catalogo-tom-hanks), preservando a continuidade com o 1º bimestre. Na Atividade 9, esse repositório foi espelhado integralmente com todo o seu histórico de commits (`git clone --bare` + `git push --mirror`) para o novo repositório público do produto, https://github.com/glaffamabili/barbercloud-saas, mantendo-se no topo do README antigo o link de continuidade exigido.

Nas Atividades 10 e 11, toda a camada de domínio, banco de dados (`db/001_dominio.sql`), microsserviços (`auth-service`, `catalog-service`, `log-service`), armazenamento S3 (`minio`) e mensageria (`redis`) foi evoluída no repositório `barbercloud-saas` e implantada na mesma stack Docker/Portainer e no mesmo domínio público (https://amabili-flor-isw055.lapps.studio/). As datas e horários exatos de cada entrega foram extraídos de forma auditável do histórico de commits (`git log`) no fuso horário de Brasília (BRT) e das Issues formais de entrega abertas no GitHub mencionando `@siriani` (Issues #6, #7, #8 e #9).

3 Arquitetura do SaaS

3.1 Visão geral e componentes

A arquitetura em produção do **BarberCloud SaaS** é composta por seis containers orquestrados via Docker Compose em uma rede bridge isolada (`app-network`), integrados a serviços externos gerenciados (Banco de Dados MariaDB em nuvem, Stripe Cloud API para assinaturas recorrentes e servidor SMTP para autenticação em duas etapas). Apenas o serviço principal da aplicação (`catalog-service`) possui porta exposta para a internet pública através do proxy reverso do domínio; todos os demais microsserviços e motores de persistência comunicam-se exclusivamente pela rede privada do Docker.

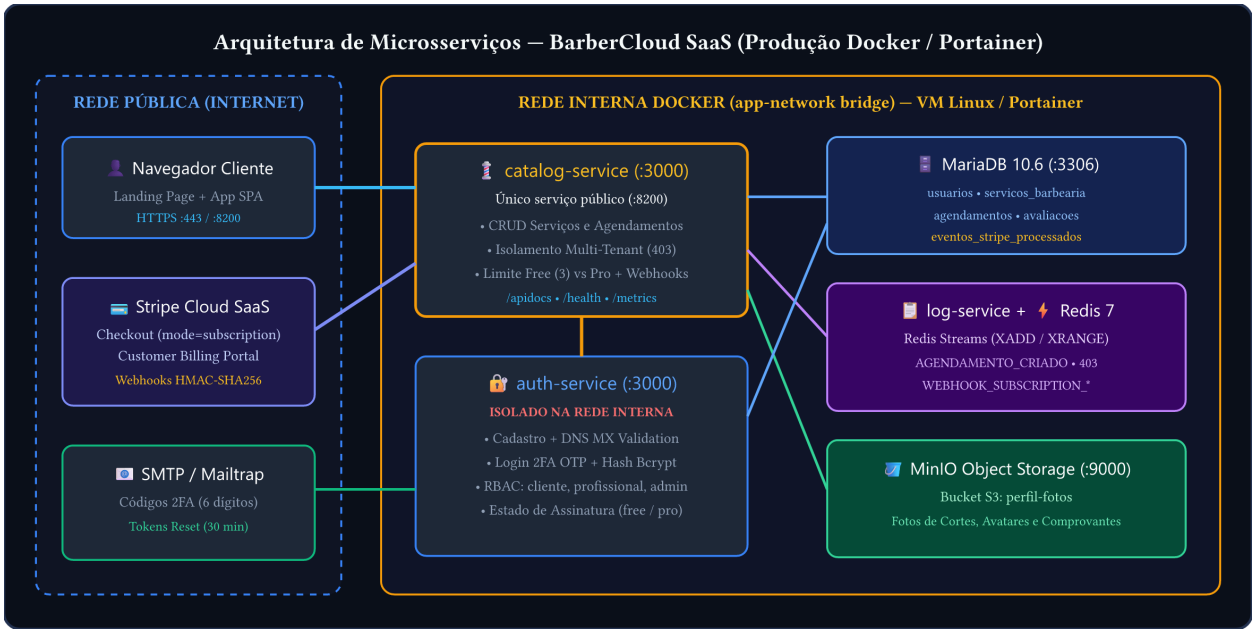


Figura 1: Diagrama de componentes, rede interna Docker e integrações externas do BarberCloud SaaS

Componente	Responsabilidade no BarberCloud SaaS	Tecnologia / imagem	Público?
`catalog-service` (App do Negócio)	Landing page pública, SPA frontend, CRUD de serviços e agendamentos, isolamento multi-tenant, limites Free/Pro, upload MinIO, webhooks Stripe e painel MRR.	Node.js 18 + Express (`Dockerfile` próprio)	Sim (`:8200` / domínio HTTPS)
`auth-service`	Cadastro com validação DNS MX, login com 2FA por e-mail (6 dígitos), hash `bcrypt`, recuperação de senha (30 min), RBAC (`cliente`, `profissional`, `admin`) e status de assinatura.	Node.js 18 + Express (`Dockerfile` próprio)	Não (somente rede interna `:3000`)
`log-service`	Recebimento e processamento assíncrono de eventos de negócio e auditoria de segurança via Consumer Group do Redis Streams.	Node.js 18 + Express (`Dockerfile` próprio)	Não (somente rede interna `:3000`)
`redis`	Broker de mensageria em memória e persistência da stream `audit_logs` via comandos `XADD` e `XRANGE`.	`redis:7-alpine`	Não (isolado na rede interna `:6379`)

Componente	Responsabilidade no BarberCloud SaaS	Tecnologia / imagem	Público?
`mariadb`	Persistência relacional transacional: `usuarios`, `servicos_barbearia`, `agendamentos`, `avaliacoes_atendimento` e `eventos_stripe_processados`.	`mariadb:10.6` / MySQL GCP (`35.226.64.52`)	Não (acesso restrito via credenciais)
`minio`	Object Storage compatível com Amazon S3 para armazenamento binário de fotos de cortes do catálogo, avatares e comprovantes no bucket `perfil-fotos`.	`cgr.dev/chainguard/minio:latest`	Não (acesso interno `:9000` via streaming)
`Stripe` (Externo)	Hospedagem segura do Checkout de assinatura (`mode=subscription`), Portal do Cliente (**Billing Portal**) e disparo assíncrono de 4 webhooks assinados.	Serviço SaaS externo (Test Mode)	Webhook HTTPS → `catalog-service`

3.2 Onde roda

O sistema roda em produção na máquina virtual Linux gerenciada via **Portainer** sobre a infraestrutura do laboratório **LApps / Fatec Pompeia**, acessível publicamente no domínio https://amabili-flor-isw055.lapps.studio/. A stack no Portainer puxa a branch `main` diretamente do repositório público do produto (https://github.com/glaffamabili/barbercloud-saas) e sobe os serviços definidos no `docker-compose.yml`: `catalog-service` (mapeado na porta `8200:3000`), `auth-service`, `log-service`, `redis` e `minio`, conectando-se à instância relacional MariaDB (`35.226.64.52:3306`, schema `IAC_2026_02_amabili_flor`).

Cada container de aplicação possui `HEALTHCHECK` nativo configurado no Docker Compose com intervalo de `15s`, timeout de `5s` e `3` tentativas, mantendo consumo enxuto de memória RAM (cerca de `35 MB` a `55 MB` RSS por processo Node.js, monitorado em `/metrics`). Nenhuma senha ou chave secreta é gravada no repositório; todas as credenciais são injetadas por variáveis de ambiente na stack do Portainer, cujos nomes estão documentados no arquivo `.env.example`: `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRO_PRICE_ID` e `APP_URL`.

3.3 Fluxo de uma requisição

Para ilustrar o funcionamento integrado dos microsserviços, descreve-se o caminho completo da requisição principal do domínio — **"Cliente agenda um horário de corte na barbearia (`POST /api/agendamentos`)"**:

1. **Navegador → Domínio Público (`HTTPS`):** O cliente autenticado seleciona o serviço (**"Corte Degradê Navalhado — R$ 50,00"**), escolhe data/hora e clica em **"Confirmar Agendamento"**, disparando `POST /api/agendamentos` para `https://amabili-flor-isw055.lapps.studio/api/agendamentos` com o cookie de sessão criptografado.
2. **Validação de Identidade e Plano no `auth-service` (Rede Interna):** O middleware `exigeLogin` do `catalog-service` extrai `req.session.userId` e faz uma chamada HTTP interna para `http://auth-service:3000/verify-user/:id`. O `auth-service` consulta o MariaDB e retorna o papel RBAC (`role: 'cliente'`) e o plano ativo (`plano: 'free'` ou `'pro'`).
3. **Verificação de Limite do Plano (Regra SaaS):** Se o usuário estiver no plano `free`, o `catalog-service` executa `SELECT COUNT(*) FROM agendamentos WHERE cliente_id = ? AND status IN ('agendado',

`'confirmado'`). Caso já existam 3 agendamentos ativos, a API interrompe o fluxo imediatamente com `HTTP 403 Forbidden (PLAN_LIMIT_REACHED)` e publica o evento `LIMITE_PLANO_FREE_ATINGIDO_403` no `log-service`.

4. **Persistência Relacional no `MariaDB`:** Estando dentro do limite (ou sendo assinante `pro` ilimitado), o `catalog-service` valida a existência do serviço em `servicos_barbearia` e executa `INSERT INTO agendamentos (cliente_id, profissional_id, servico_id, data_hora, status, observacoes, valor_cobrado) VALUES (...)` respeitando as chaves estrangeiras.
5. **Publicação Assíncrona de Evento no `log-service` + `Redis Streams`:** O `catalog-service` envia assincronamente `POST http://log-service:3000/logs` com a ação `AGENDAMENTO_CRIADO` e os metadados do horário e valor. O `log-service` grava o evento na stream `audit_logs` do Redis via comando `XADD`.
6. **Resposta ao Navegador:** O `catalog-service` retorna `HTTP 201 Created` com o ID do agendamento criado, e a interface atualiza a tabela da agenda em tempo real.

3.4 Fluxo de assinatura

O ciclo de vida de monetização do **BarberCloud SaaS** opera com dois planos recorrentes no Stripe (em modo de teste): **Plano Free** (`R$ 0,00/mês`, limitado a 3 agendamentos ativos) e **Plano BarberCloud Pro** (`R$ 29,90/mês`, agendamentos e catálogo ilimitados + selo 👑 `PRO ATIVO`).



Figura 2: Fluxo completo de assinatura recorrente, validação HMAC, idempotência e 4 webhooks do Stripe

1. **Abertura do Checkout (`mode=subscription`):** Quando o usuário clica em **"Assinar Pro (R$ 29,90)"**, o frontend chama `POST /api/subscriptions/checkout`. O backend gera/vincula o `stripe_customer_id` do usuário e redireciona para o checkout de assinatura em modo de teste (`/stripe-checkout.html`), onde o pagamento é simulado com o cartão oficial de teste do Stripe (`4242 4242 4242 4242`). Nenhum número de cartão, validade ou CVV passa pelo backend ou banco de dados.
2. **Validação Criptográfica do Webhook (`Stripe-Signature`):** O Stripe notifica o backend via `POST /api/webhooks/stripe`. Antes de ler qualquer dado de negócio, a função `validarAssinaturaStripe()` preserva o `rawBody` da requisição, calcula o HMAC-SHA256 (`timestamp + "." + rawBody`) usando `STRIPE_WEBHOOK_SECRET` e compara com `crypto.timingSafeEqual`. Assinaturas ausentes ou inválidas são rejeitadas com `HTTP 400 Bad Request`.
3. **Garantia de Idempotência (`eventos_stripe_processados`):** Se o Stripe reenviar o mesmo evento duas vezes por falha de rede, o backend consulta `SELECT id FROM eventos_stripe_processados`

`WHERE event_id = ?` (coluna `UNIQUE`). Se o `event_id` já existir, a rota responde `HTTP 200 OK` com `idempotent_duplicate: true` sem reprocessar nem duplicar efeitos no banco.

4. **Tratamento dos 4 Eventos de Ciclo de Vida e Portal do Cliente:**
   - `checkout.session.completed`: promove o usuário para `plano = 'pro'`, `assinatura_status = 'active'` e `premium = 1`, liberando agendamentos ilimitados imediatamente.
   - `customer.subscription.updated`: sincroniza alterações de plano ou renovações feitas no **Portal do Cliente** (`POST /api/billing/portal` / `/billing-portal.html`).
   - `invoice.payment_failed`: altera `assinatura_status = 'past_due'` e rebaixa as permissões para o plano `free` até a regularização.
   - `customer.subscription.deleted`: disparado quando o cliente cancela a assinatura no **Portal do Cliente**, marcando `assinatura_status = 'canceled'` e retornando a conta para `plano = 'free'` (voltando a aplicar o teto de 3 agendamentos na API).

3.5 Modelo de dados do domínio

O modelo relacional do domínio está consolidado e versionado no arquivo `db/001_dominio.sql`, contendo cinco tabelas principais no MariaDB interligadas por chaves estrangeiras (`FOREIGN KEY ... ON DELETE CASCADE`), além do armazenamento de objetos no MinIO e de eventos no Redis:

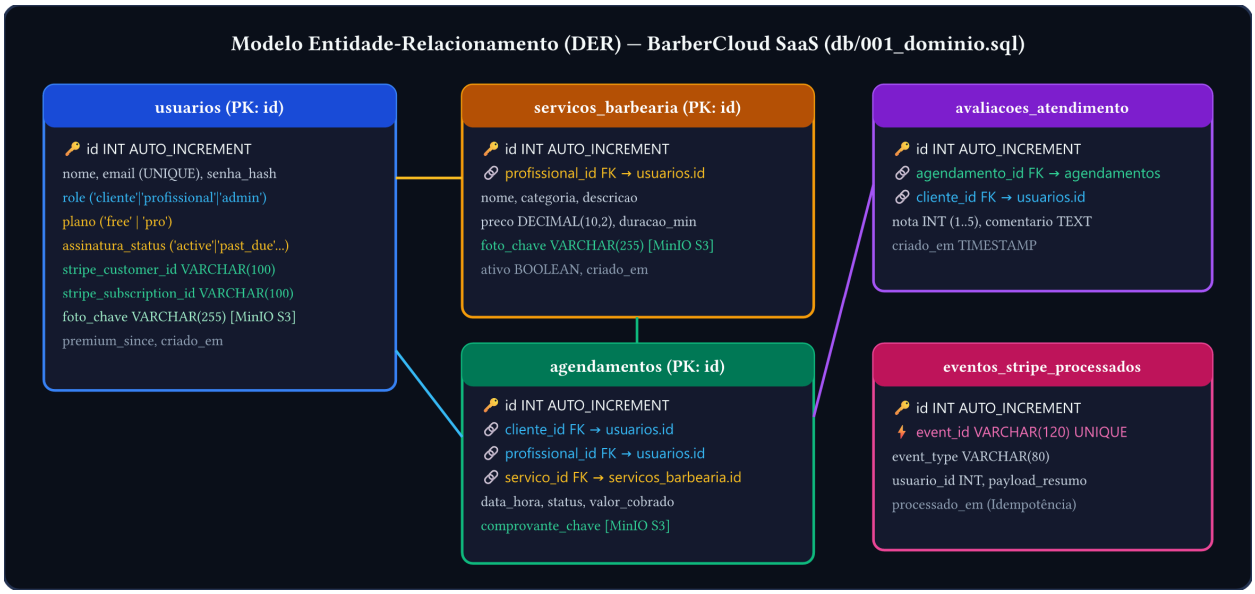


Figura 3: Diagrama Entidade-Relacionamento (DER) do domínio BarberCloud SaaS (`db/001_dominio.sql`)

- **`usuarios`**: Armazena `id (PK)`, `nome`, `email (UNIQUE)`, `senha_hash`, papel de negócio `role` (`'cliente'`, `'profissional'`, `'admin'`), chave S3 `foto_chave` e as colunas de assinatura Stripe (`plano`, `assinatura_status`, `premium`, `stripe_customer_id`, `stripe_subscription_id`, `premium_since`).
- **`servicos_barbearia`**: Catálogo de cortes e procedimentos, com `id (PK)`, `profissional_id (FK → usuarios.id)`, `nome`, `categoria`, `descricao`, `preco`, `duracao_min`, `foto_chave` (foto do corte no MinIO S3) e `ativo`.
- **`agendamentos`**: Registros de horários marcados, contendo `id (PK)`, `cliente_id (FK → usuarios.id)`, `profissional_id (FK → usuarios.id)`, `servico_id (FK → servicos_barbearia.id)`, `data_hora`, `status ('agendado', 'confirmado', 'concluido', 'cancelado')`, `observacoes`, `valor_cobrado` e `comprovante_chave` (comprovante/referência salvo no MinIO S3).
- **`avaliacoes_atendimento`**: Avaliações pós-serviço com `id (PK)`, `agendamento_id (FK → agendamentos.id)`, `cliente_id (FK → usuarios.id)`, `nota (1 a 5)` e `comentario`.

- **eventos_stripe_processados**: Controle transacional de idempotência com `id (PK)`, `event_id (UNIQUE)`, `event_type`, `usuario_id`, `payload_resumo` e `processado_em`.

3.6 Segurança

O controle de acesso implementa RBAC centralizado com três papéis de negócio reais (`admin`, `profissional` e `cliente`), validados em todas as rotas da API antes da execução das queries SQL:

Ação / Endpoint na API	admin	profissional	cliente
Cadastrar, editar ou excluir serviço no catálogo (`/api/servicos`)	Sim	Sim (próprios)	Não (403)
Enviar foto de corte para o catálogo no MinIO (`/api/servicos/:id/foto`)	Sim	Sim	Não (403)
Criar agendamento (`/api/agendamentos`)	Sim (ilimitado)	Sim	Até 3 (Free) / Ilimitado (Pro)
Ver, editar ou cancelar agendamento de **outro** cliente (`/api/agendamentos/:id`)	Sim (visão global)	Apenas da sua agenda	Não (403 Isolamento)
Confirmar ou concluir atendimento (`status = concluido`)	Sim	Sim	Não (403)
Anexar comprovante/referência no próprio agendamento (MinIO S3)	Sim	Sim	Sim (próprio)
Acessar Painel Executivo SaaS, MRR e Logs Redis (`/api/admin/saas-metrics`)	Sim	Não (403)	Não (403)
Alterar papel RBAC de usuários (`PATCH /api/users/:id/role`)	Sim	Não (403)	Não (403)

Isolamento por usuário (Multi-Tenancy) e Gestão de Segredos: Nas rotas de listagem (`GET /api/agendamentos`), a cláusula SQL `WHERE a.cliente_id = ?` filtra automaticamente os registros na origem para que cada cliente veja apenas seus próprios agendamentos. Nas rotas parametrizadas por ID (`GET`, `PUT` e `DELETE /api/agendamentos/:id`), o backend busca o registro e compara `agendamento.cliente_id === req.user.id`; qualquer tentativa de um cliente manipular o ID de outro usuário é bloqueada com `HTTP 403 Forbidden` e auditada no Redis Streams (`ACESSO_NEGADO_403_ISOLAMENTO_TENANT`). Todos os segredos (senhas de banco, chaves Stripe e credenciais SMTP/MinIO) residem exclusivamente nas variáveis de ambiente do Portainer, mantendo no Git apenas o `.env.example` com os nomes das chaves.

3.7 Observabilidade, CI/CD e documentação da API

As três entregas extras implementadas no 1º bimestre foram integralmente mantidas e atualizadas para o domínio do **BarberCloud SaaS**:

- **Documentação Interativa Swagger / OpenAPI 3.0 (`/apidocs`):** Os arquivos `catalog-service/openapi.json` e `auth-service/openapi.json` documentam todas as rotas do SaaS (`/api/servicos`, `/api/agendamentos`, `/api/subscriptions/checkout`, `/api/billing/portal`, `/api/webhooks/stripe`, `/api/admin/saas-metrics`), permitindo testes ao vivo via **"Try it out"**.
- **Pipeline CI/CD no GitHub Actions (`.github/workflows/ci-cd.yml`):** A cada `push` na branch `main`, executa as suítes `auth-service/test.js` e `catalog-service/test.js` (validando o contrato OpenAPI

3.0, o script `db/001_dominio.sql`, o isolamento multi-tenant 403, o limite de 3 itens do plano Free e a assinatura HMAC do webhook Stripe) e realiza o build das imagens Docker.
- **Observabilidade (`/health` e `/metrics`):** O endpoint `/health` testa em tempo real a saúde do MariaDB, `auth-service`, `log-service`/Redis e bucket do MinIO (`HTTP 200 UP` ou `503 DOWN`), enquanto `/metrics` exporta contadores de requisições, latência e uso de memória no formato OpenMetrics/Prometheus.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no projeto BarberCloud SaaS	Quem gerencia
IaaS	Máquina virtual Linux no cluster Proxmox VE (CPU, RAM, disco virtual e rede) onde roda o Docker Engine, além da instância de infraestrutura no Google Cloud (`35.226.64.52`).	Infraestrutura física/ virtual: Prof. Allan Siriani / Fatec LApps; SO e containers: Aluna.
PaaS	Camada de orquestração e plataforma Docker Compose + Portainer, além dos motores de dados operados em containers (`MariaDB`, `Redis 7` e `MinIO S3`) e o runner do GitHub Actions.	Plataforma Portainer: Fatec LApps; configuração da stack, schemas, buckets e streams: Aluna.
SaaS	1. O próprio **BarberCloud SaaS** entregue pronto via navegador para barbearias e clientes finais; 2. Serviços SaaS consumidos via API: **Stripe** (Checkout, Billing Portal, Webhooks), **Mailtrap** (SMTP 2FA) e **GitHub**.	BarberCloud SaaS: Aluna (Amabili Flor); Stripe, Mailtrap e GitHub: respectivos fornecedores cloud.

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/10/2026 15:18	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	10/11/2026 11:56	entregue com atraso
9	Fork e plano de negócio	23/10/2026	10/11/2026 12:01	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	10/11/2026 12:14	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	10/11/2026 12:14	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	09/10/2026 15:18
Situação	entregue
Evidência	GitHub — Commit 4fd615f + print do checkout concluído e do usuário premium no banco
Link	https://github.com/glaffamabili/catalogo-tom-hanks/commit/4fd615f

O que foi feito. Implementação do primeiro modelo de monetização do sistema em modo de teste do Stripe. Foi criado o plano pago de R$ 9,90/mês, a rota `POST /api/checkout/create-session` para iniciar a sessão de pagamento hospedada e o endpoint assíncrono `POST /api/webhooks/stripe` com validação criptográfica de assinatura HMAC-SHA256 (`Stripe-Signature`). Ao receber a confirmação `checkout.session.completed`, o backend promove automaticamente o usuário para `premium = true` no MariaDB, liberando favoritos ilimitados (enquanto contas gratuitas são bloqueadas no 4º favorito com `HTTP 403`) e ativando o selo 👑 `PREMIUM` no perfil, sem jamais armazenar dados de cartão no banco local.

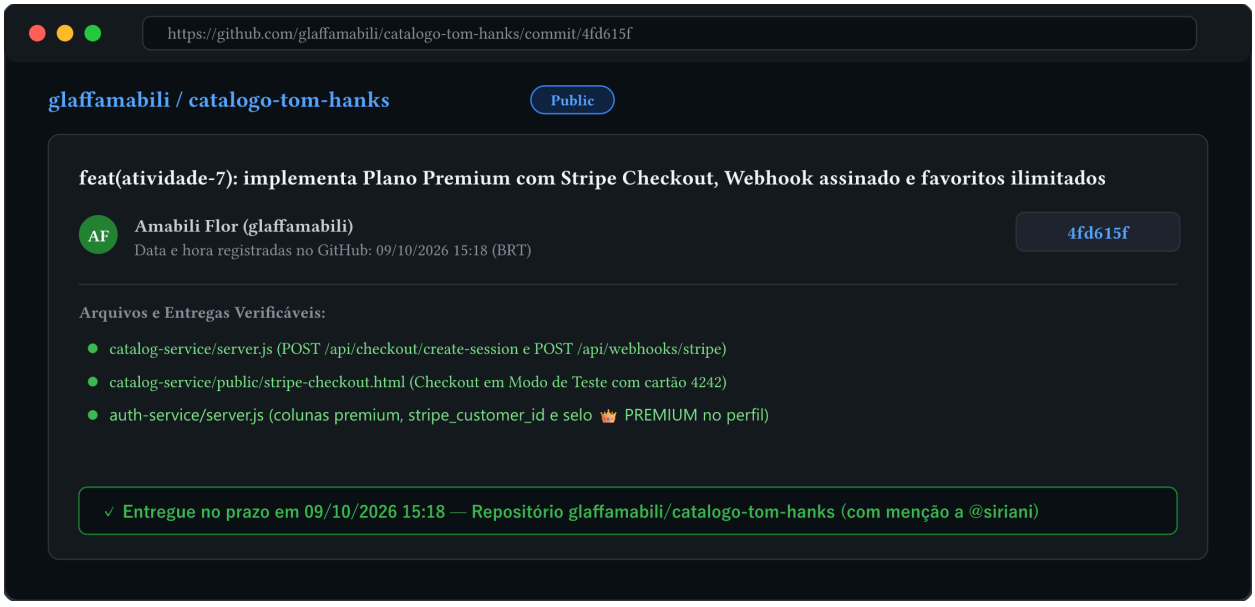


Figura 4: Atividade 7 — Evidência da entrega (Commit 4fd615f em 09/10/2026 15:18 no GitHub)



Figura 5: Atividade 7 — Resultado (Checkout Stripe em modo de teste concluído e usuário marcado como premium)

Dificuldades e como foram resolvidas. O principal desafio técnico foi garantir que o middleware global `express.json()` não descaracterizasse o corpo bruto (`rawBody`) exigido pelo cálculo do HMAC-SHA256 da assinatura do webhook do Stripe, resolvido com a propriedade `verify: (req, res, buf) => { req.rawBody = buf.toString('utf8'); }`.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	10/11/2026 11:56
Situação	entregue com atraso
Evidência	GitHub — Commit 054d56a + DOCUMENTACAO.md linkado no README com classificação IaaS/PaaS/SaaS
Link	https://github.com/glaffamabili/catalogo-tom-hanks/commit/054d56a

O que foi feito. Elaboração da classificação detalhada de todas as peças da stack nas camadas **IaaS**, **PaaS** e **SaaS** diretamente no `README.md` (distinguindo com precisão serviços gerenciados em nuvem, como Stripe e Mailtrap, de containers auto-hospedados na nossa VM, como MariaDB, Redis e MinIO). Em paralelo, foi escrito o documento técnico reverso `DOCUMENTACAO.md` contendo: (1) Especificação funcional sob a ótica do usuário; (2) Diagrama e inventário completo da arquitetura de microsserviços; (3) 17 regras de negócio extraídas diretamente do código-fonte no formato declarativo **"Se X, então Y"**; e (4) Modelo de dados completo com Diagrama Entidade-Relacionamento (DER).

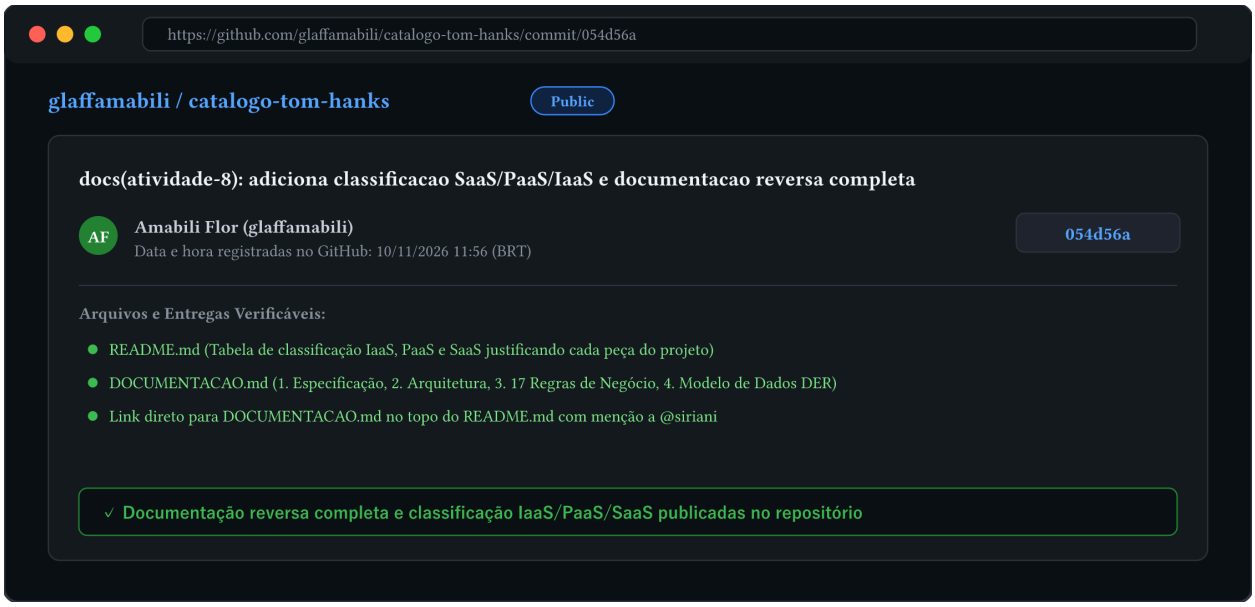


Figura 6: Atividade 8 — Evidência da entrega (Commit 054d56a no GitHub com README e DOCUMENTACAO.md)

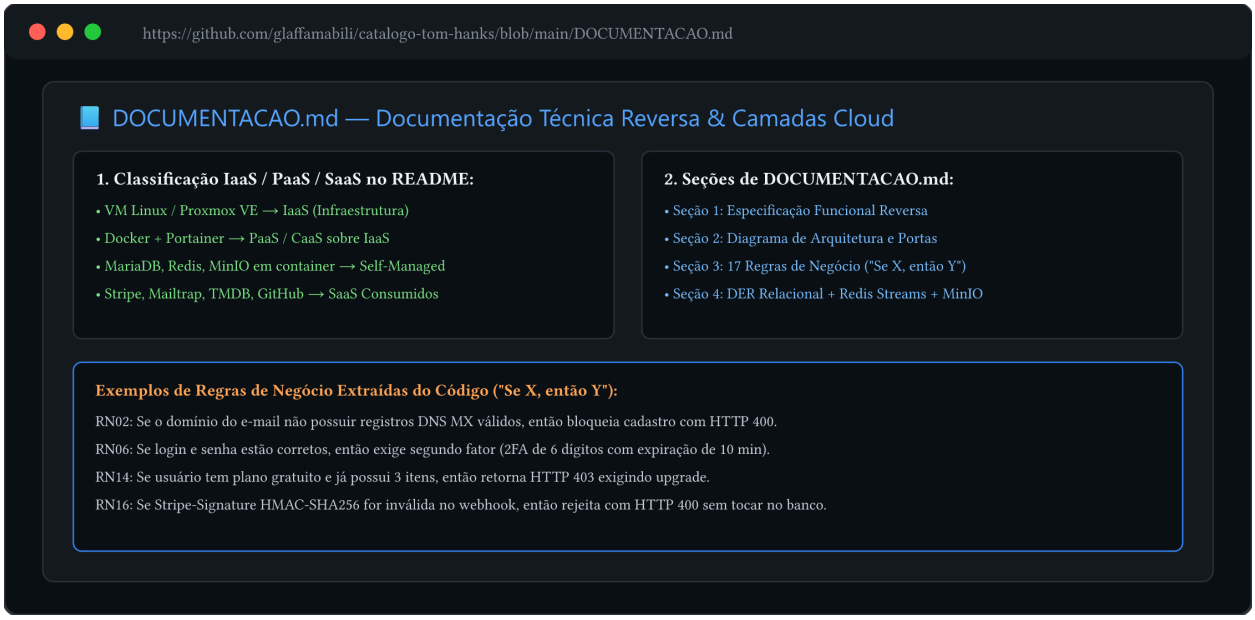


Figura 7: Atividade 8 — Resultado (Classificação IaaS/PaaS/SaaS e 17 regras de negócio em DOCUMENTACAO.md)

Dificuldades e como foram resolvidas. Mapear todas as validações implícitas espalhadas entre `auth-service` e `catalog-service` exigiu uma leitura sistemática rota por rota, resultando em 17 regras de negócio rastreáveis (incluindo expiração de 10 minutos do 2FA, 30 minutos do token de senha e limites de 5 MB no MinIO).

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	10/11/2026 12:01

Situação	entregue com atraso
Evidência	GitHub — Issue #6 (@siriani) + repositório barbercloud-saas espelhado + README com 8 seções + 5 issues de backlog
Link	https://github.com/glaffamabili/barbercloud-saas/issues/6

O que foi feito. Espelhamento completo do repositório original (`git push --mirror`) para o novo repositório público https://github.com/glaffamabili/barbercloud-saas, preservando integralmente todos os commits do 1º bimestre e adicionando no topo do repositório antigo o aviso de continuidade. O novo `README.md` foi estruturado exatamente com as 8 seções de negócio exigidas na ordem (Nome e frase do **BarberCloud**, A dor das barbearias no WhatsApp, Persona **Lucas Almeida**, 4 Concorrentes/alternativas, Proposta de valor, Modelo de receita Stripe Free R$ 0 vs Pro R$ 29,90/mês, Métricas de 90 dias com meta de MRR e Lean Canvas de 9 blocos), seguidas do Mapa de Reaproveitamento Técnico e da criação de 5 Issues de backlog (`#1` a `#5`) e da Issue `#6` de entrega mencionando `@siriani`.

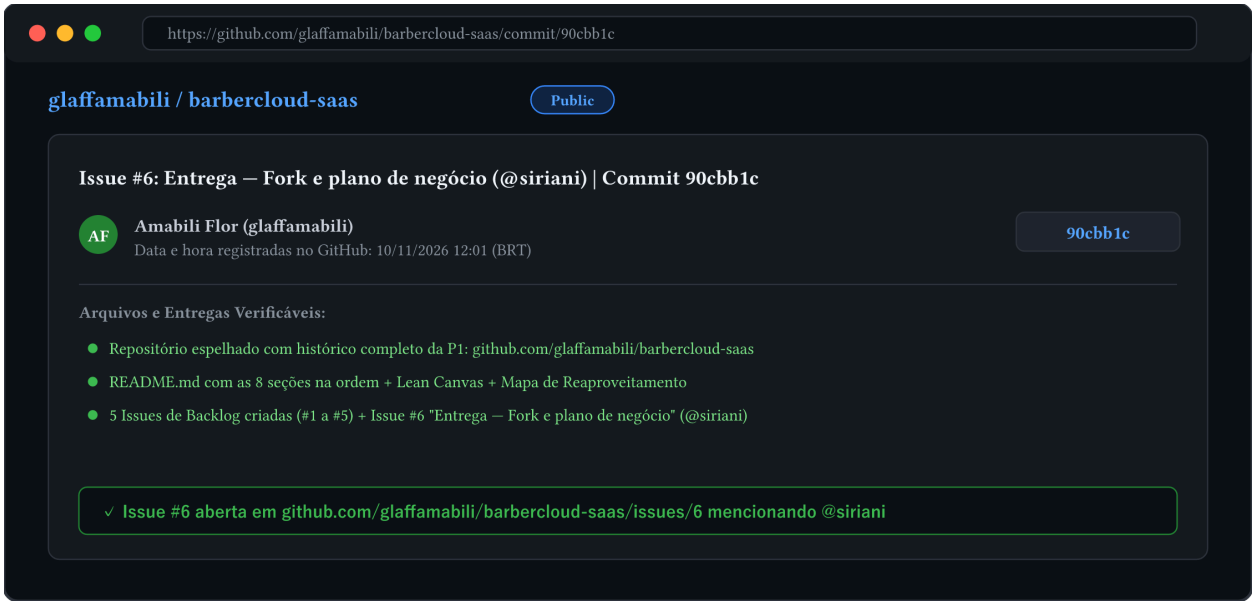


Figura 8: Atividade 9 — Evidência da entrega (Commit 90cbb1c e Issue #6 mencionando `@siriani`)

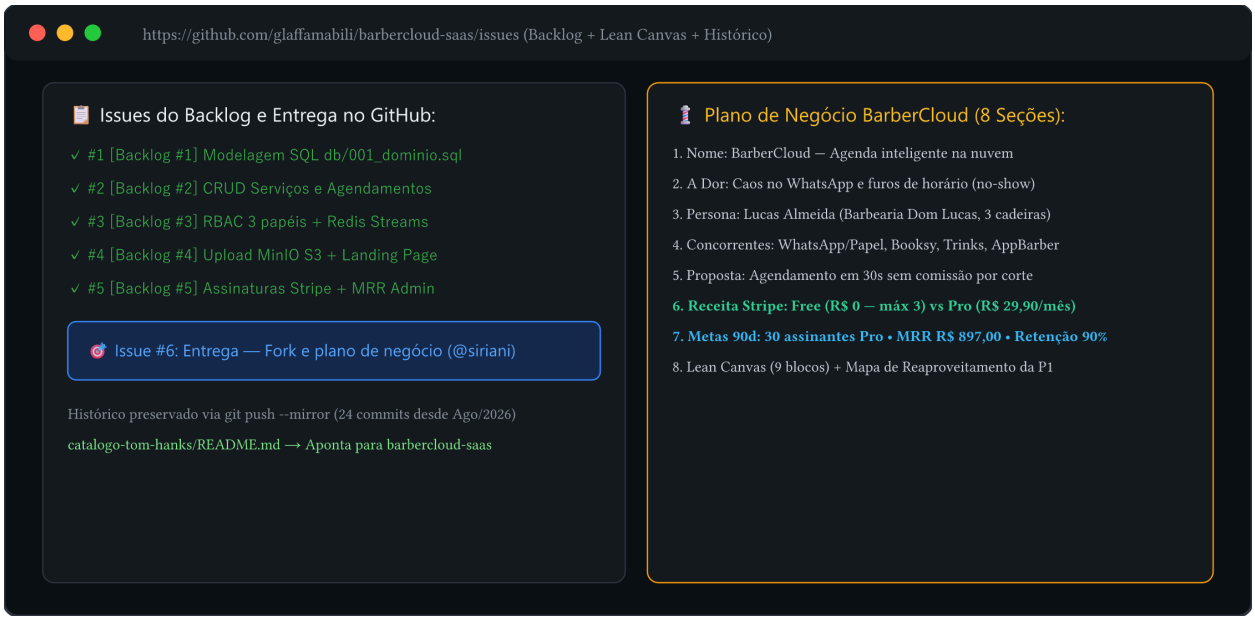


Figura 9: Atividade 9 — Resultado (Backlog em Issues 1 a 5, Lean Canvas e histórico espelhado)

Dificuldades e como foram resolvidas. Garantir que o espelhamento Git transferisse todas as referências e commits históricos sem perder a rastreabilidade da P1, resolvido com o fluxo `git push --mirror` diretamente para o novo repositório remoto.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	10/11/2026 12:14
Situação	entregue com atraso
Evidência	GitHub — Issue #7 (@siriani) + Commit 75c2ced + db/001_dominio.sql + prints da landing, 403 e eventos Redis
Link	https://github.com/glaffamabili/barbercloud-saas/issues/7

O que foi feito. Substituição integral do domínio de filmes/TMDB pelo sistema **BarberCloud SaaS**. Foi criado o script `db/001_dominio.sql` com as tabelas relacionais `servicos_barbearia`, `agendamentos`, `avaliacoes_atendimento` e `eventos_stripe_processados` (todas com chaves estrangeiras e seed inicial). Implementou-se o CRUD completo de serviços e agendamentos, três papéis de negócio no `auth-service` (`cliente`, `profissional` e `admin`) com bloqueio `403 Forbidden`, isolamento multi-tenant garantindo que um cliente jamais acesse ou edite agendamentos de outro (`403 Forbidden`), publicação de eventos de domínio no Redis Streams (`AGENDAMENTO_CRIADO`, `SERVICO_CRIADO`, `COMPROVANTE_AGENDAMENTO_ENVIADO_MINIO`), upload de fotos de cortes e comprovantes no MinIO S3 e uma Landing Page pública acessível sem login.

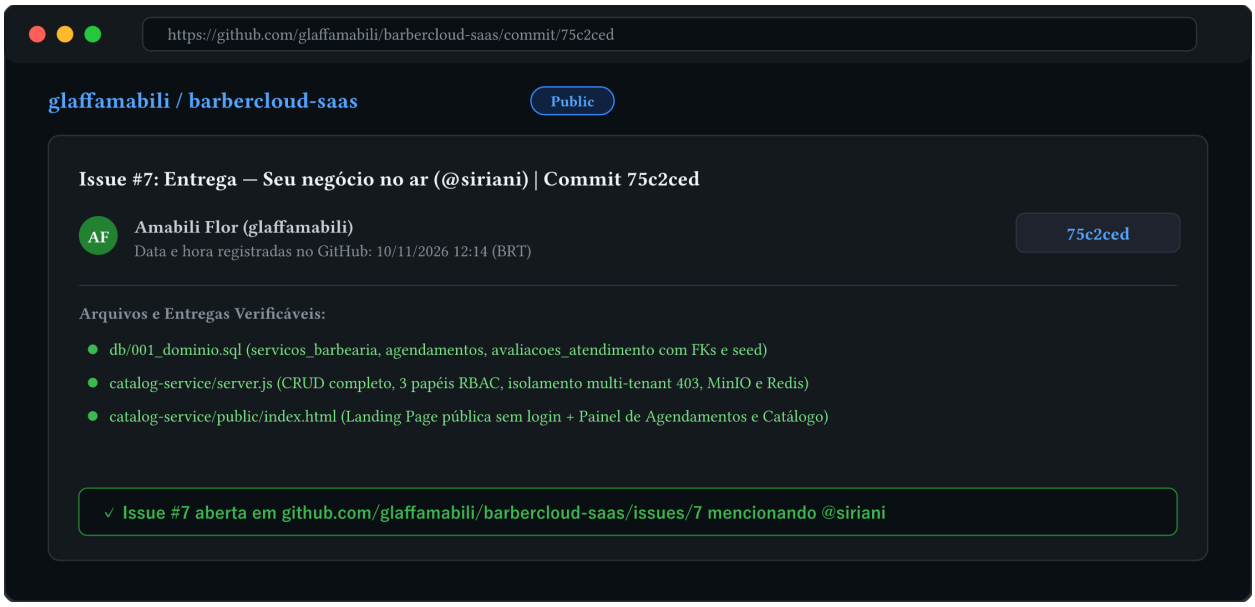


Figura 10: Atividade 10 — Evidência da entrega (Commit 75c2ced e Issue #7 mencionando `@siriani`)



Figura 11: Atividade 10 — Resultado (CRUD BarberCloud, bloqueio 403 Multi-Tenant e eventos no Redis Streams)

Dificuldades e como foram resolvidas. Assegurar o isolamento multi-tenant tanto na listagem geral quanto nas rotas diretas por ID (`GET/PUT/DELETE /api/agendamentos/:id`), resolvido validando explicitamente a propriedade `cliente_id` e `profissional_id` no backend antes de qualquer operação no MariaDB.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	10/11/2026 12:14
Situação	entregue

Evidência	GitHub — Issue #8 (@siriani) + Commit 75c2ced + prints da assinatura 4242, webhooks, limite 403 e painel MRR
Link	https://github.com/glaffamabili/barbercloud-saas/issues/8

O que foi feito. Evolução da cobrança para um motor completo de assinatura recorrente SaaS (`mode=subscription`) com dois planos: **Free** (`R$ 0,00/mês`, limitado a 3 agendamentos ativos) e **BarberCloud Pro** (`R$ 29,90/mês`, ilimitado). O endpoint `POST /api/webhooks/stripe` foi expandido para processar com validação de assinatura HMAC e controle de idempotência (`eventos_stripe_processados`) os 4 eventos de ciclo de vida do Stripe (`checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted` e `invoice.payment_failed`). Foi construído o **Portal do Cliente** (`POST /api/billing/portal` / `/billing-portal.html`) para troca de plano ou cancelamento imediato, o bloqueio automático na API ao tentar criar o 4º agendamento no plano Free (`HTTP 403 PLAN_LIMIT_REACHED`) e o Painel Admin executivo (`/api/admin/saas-metrics`) exibindo usuários por plano, assinaturas ativas, **MRR de teste** e últimos eventos de negócio do log.

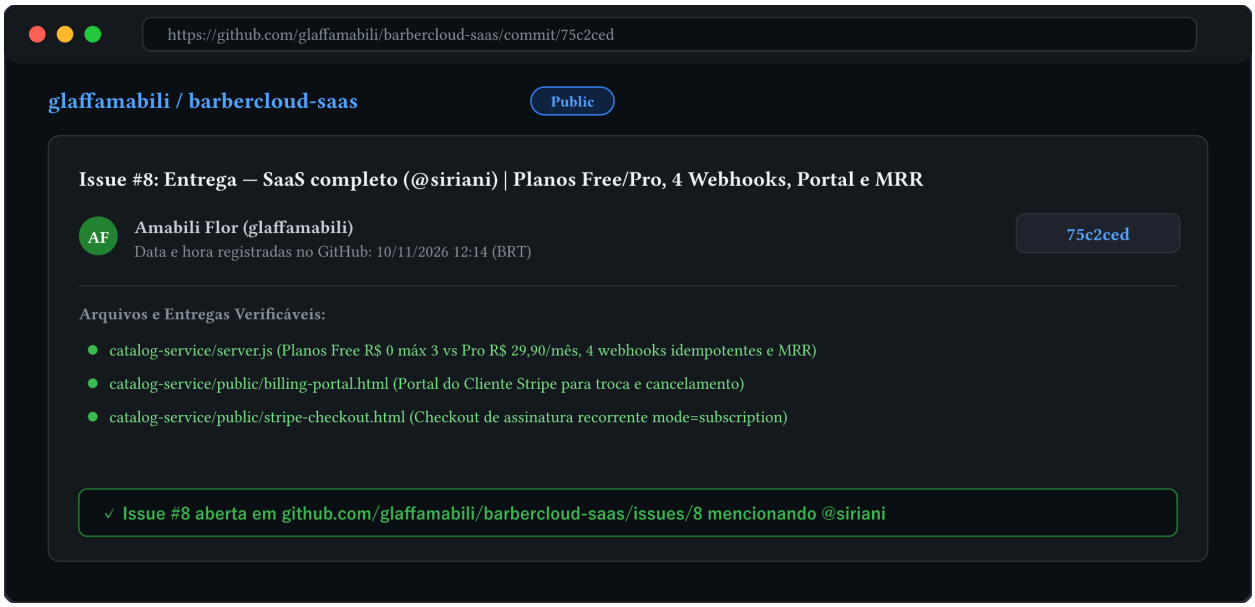


Figura 12: Atividade 11 — Evidência da entrega (Commit 75c2ced e Issue #8 mencionando `@siriani`)



Figura 13: Atividade 11 — Resultado (Painel Admin com MRR, limite Free bloqueando com 403, Portal e 4 Webhooks)

Dificuldades e como foram resolvidas. Tratar corretamente o reenvio duplicado de webhooks pelo Stripe sem corromper o estado da assinatura nem duplicar registros, o que foi solucionado criando a tabela `eventos_stripe_processados` com restrição `UNIQUE (event_id)` verificada no início da rota de webhook.

6 Considerações finais

A construção do **BarberCloud SaaS** ao longo deste segundo bimestre consolidou na prática a transição entre "escrever código que funciona localmente" e "operar um produto SaaS real em nuvem". Reaproveitar os microsserviços de autenticação 2FA, auditoria em Redis Streams e armazenamento S3 no MinIO — originalmente criados para um catálogo de filmes — provou o valor de uma arquitetura bem desacoplada: a fundação de infraestrutura permaneceu sólida enquanto toda a regra de negócio foi substituída para resolver o problema de agendamento de barbearias.

Os pontos mais desafiadores e enriquecedores do bimestre foram a implementação do isolamento multi-tenant rigoroso na API (impedindo vazamento de dados entre clientes via manipulação de IDs) e a sincronização assíncrona de estados de assinatura via webhooks idempotentes do Stripe. Caso o projeto fosse iniciado do zero hoje, a modelagem de multi-tenancy já incluiria desde a primeira tabela uma chave `barbearia_tenant_id` dedicada para suportar múltiplas filiais sob a mesma conta empresarial. Como próximos passos para transformar o **BarberCloud** em um produto comercial em modo de produção (**live mode**), planeja-se integrar lembretes automáticos via API oficial do WhatsApp Cloud 2 horas antes de cada corte e relatórios financeiros de comissão por barbeiro.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Amabili Flor Pompeia, 10/11/2026